Table of contents

Introduction **4**
C Program Structure and Examples **4**
i) Basic Hello World Program 4
ii) Program with User Input 4
iii) Program with Formatted Output 5
GCC Compilation Process **5**
i) Installing GCC on Linux 5
ii) Four Stages of Compilation 5
ii) Compilation Stage Examples 6
Return Values and Exit Status **7**
i) Understanding Return Values 7
ii) Checking Exit Status 8
Knowledge Check Questions **8**
Lab Tasks Completed **10**
Screenshot and evidence **15**
Conclusion **17**

List of Figures

Figure 1: Basic Hello World C Program....4
Figure 2: C Program with User Input....4
Figure 3: C Program with Formatted Output....5
Figure 4: GCC Installation Commands....5
Figure 5: Preprocessing Stage....6
Figure 6: Compilation Stage....6
Figure 7: Assembly Stage....7
Figure 8: Linking Stage....7
Figure 9: Program with Return Value....7
Figure 10: Checking Program Exit Status....8
Figure 11: C program with integer input/output....9
Figure 12: Output of C program....9
Figure 13: Linux command to create .c file and list files inside a folder....10
Figure 14: Linux command to write and view contents inside .c file. Also C language program to print Hello, World! and return 0....10
Figure 15: Linux command for 4 compilation stages with its output files. Linux command to run the program to view the output and return value....11
Figure 16: C language program modified to return 1 and print "Program executed unsuccessfully".. 11
Figure 17: Linux command to view the output and return value that was updated....12
Figure 18: C language program to accept user input and display it....12
Figure 19: Linux command to view the output and return value....12
Figure 20: Linux command for 4 compilation stages with its output files. Linux command to run the program to view the output and return value....13
Figure 21: Linux command for 4 compilation stages with its output files. Linux command to run the program to view the output and return value....13
Figure 22: C language program that returns integer value 50....14
Figure 23: Linux command for 4 compilation stages with its output files. And linux command to run the program to view the output and return value....14

List of Tables

Table 1: The Four Stages of C Compilation.....6

Introduction

This report documents the practical work completed for Lecture2 of ST5003CMD Operating Systems, Security and Networks.The focus is on understanding C program structure, compilation stages, and basic Linux commands for software development.

C Program Structure and Examples

i) Basic Hello World Program

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:

```c
#include <stdio.h>

int main() {
        printf("Hello!\n");
        return 1;
}
```

Figure 1: Basic Hello World C Program.

ii) Program with User Input

This example demonstrates the use of printf and scanf for input/output operations:

```c
#include <stdio.h>
int main() {
        int number;
        printf("Enter a number: ");
        scanf("%d", &number);
        printf("You entered: %d\n", number);
        return 0;
}
```

Figure 2: C Program with User Input.

iii) Program with Formatted Output

Demonstrating multiple format specifiers and variables:

```c
#include <stdio.h>

int main() {
        char name[50];
        int age;
        float height;

        printf("Enter your name: ");
        scanf("%s", name);

        printf("Enter your age: ");
        scanf("%d", &age);

        printf("Enter your height (m): ");
        scanf("%f", &height);

        printf("\n--- User Info ---\n");
        printf("Name: %s\n", name);
        printf("Age: %d years\n", age);
        printf("Height: %.2f m\n", height);

        return 0;
}
```

Figure 3: C Program with Formatted Output.

GCC Compilation Process

i) Installing GCC on Linux

The following commands are used to install and verify GCC:

```
root@DESKTOP-OAP8OK0:~# apt install build-essential
root@DESKTOP-OAP8OK0:~# gcc --version
```

Figure 4: GCC Installation Commands

ii) Four Stages of Compilation

The C compilation process consists of four distinct stages. Each stage transforms the source code closer to an executable program.

Stage	Command	Output	Purpose
1. Preprocessing	gcc -E hello.c -o hello.i	.i file	Expand headers & macros
2. Compiling	gcc -S hello.i -o hello.s	.s file	Generate Assembly code
3. Assembling	gcc -C hello.s -o hello.o	.o file	Create object file
4. Linking	gcc hello.o -o hello	executable	Final executable

Table 1: The Four Stages of C Compilation.

ii) Compilation Stage Examples

1) Stage 1: Preprocessing:

```
# Expand all headers and macros
gcc -E hello.c -o hello.i

# View the preprocessed output
cat hello.i | less
```

Figure 5: Preprocessing Stage.

2) Stage 2: Compilation to Assembly:

```
# Generate assembly code
gcc -S hello.i -o hello.s

# View assembly code
cat hello.s
```

Figure 6: Compilation Stage.

3) Stage 3: Assembly to Object File:

```
# Create object file (machine code)
gcc -c hello.s -o hello.o

# Check file type
file hello.o
```

Figure 7: Assembly Stage.

4) Stage 4: Linking:

```
# Create final executable
gcc hello.o-o hello

# Run the program
./hello
```

Figure 8: Linking Stage.

Return Values and Exit Status

i) Understanding Return Values

The return value of main() indicates program success or failure to the OS:

```c
#include <stdio.h>
int main() {
    printf("Program executed successfully\n");
    return 0; // 0 indicates success
}
```

Figure 9: Program with Return Value.

ii) Checking Exit Status

```
# Run program
./hello

# Check exit status (returns 0 for success)
echo $?

# Try with return 1
# Modify program to return 1, recompile, and check:
echo $? # Should return 1 (indicating error)
```

Figure 10: Checking Program Exit Status.

Knowledge Check Questions

Q1. What is the purpose of the #include directive in C programming? Provide an example of its usage.

Ans: The purpose of #include directive in C programming is it tells the compiler to bring in code from a library (like stdio.h) so you can use it's functions such as printf and scanf.

Example:
#include <stdio.h>

Q2. Explain the difference between the four stages of compilation. What file extension is produced at each stage?

Ans: The four stages of compilation with it's produced file extension and purpose:

- Preprocessing (.i) → expands headers/macros.
- Compilation (.s) → generates assembly code.
- Assembling (.o) → converts assembly into machine code.
- Linking (executable) → combines object files into a runnable program.

Q3. What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

Ans: The return value of 0 and non-zero(1) from the main() function indicates to the operating system is:

- return 0; → success.

- Non-zero (like return 1;) → error/failure.

Q4. Write a C program that:

- Declares an integer variable.
- Asks the user to input a value.
- Displays the entered value back to the user.

Ans:

```c
#include <stdio.h>
int main() {
        int number;
        printf("Enter a number: ");
        scanf("%i", &number);
        printf("You entered: %i\n", number);
        return 0;
}
```

Figure 11: C program with integer input/output.

```
Enter a number: 5
You entered: 5
```

Figure 12: Output of C program.

Note: We can put %i or %d. The similarity is that both print an integer in decimal(base 10), while the difference is %i can print an integer in decimal(base 10), octal(base 8) and hexadecimal(base 16), but %d can only print an integer in decimal(base 10).

Q5. What GCC command would you use to:

1) Only preprocess a file without compiling?

2) Generate assembly code but not assemble it?

3) Create an object file without linking?

Ans: The GCC command I would use:

1. Preprocess only is → gcc -E hello.c -o hello.i

2. Generate assembly only is → gcc -S hello.i -o hello.s
3. Create object file only is → gcc -c hello.s -o hello.o

Q6. Explain why C is called a ”compiled language” and how this differs from interpreted languages.

Ans: C is a compiled language because source code (any code/program we write and save in a “.c” file) is transformed into machine code before running.

The Interpreted languages differ from the C language because Interpreted languages (like Python) run line-by-line without creating machine code first.

Lab Tasks Completed

1) **Task 1:** Compiled hello.c with return 0 and verified exit status using echo $?
Ans:

```
root@DESKTOP-OAP8OK0:/batch38# touch hello.c
root@DESKTOP-OAP8OK0:/batch38# ls
hello.c
```

Figure 13: Linux command to create .c file and list files inside a folder.

```
root@DESKTOP-OAP8OK0:/batch38/Week2# nano hello6.c
root@DESKTOP-OAP8OK0:/batch38/Week2# cat hello6.c
#include <stdio.h>
int main() {
    printf("Program executed successfully\n");
    return 0; // 0 indicates success
}
```

Figure 14: Linux command to write and view contents inside .c file. Also C language program to print Hello, World! and return 0.

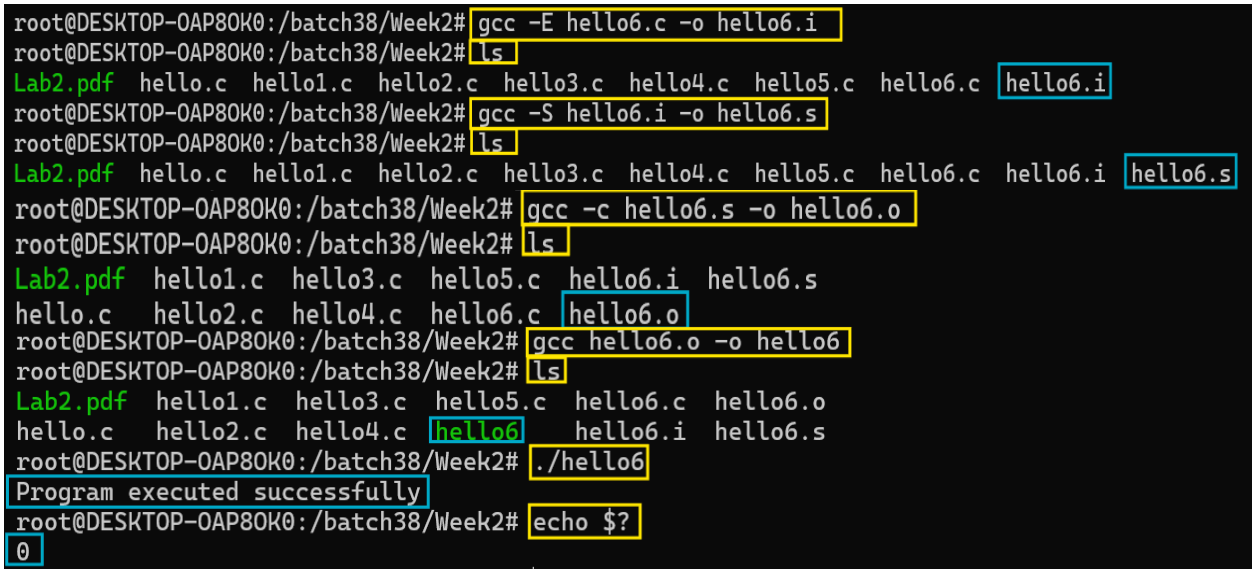


Figure 15: Linux command for 4 compilation stages with its output files. Linux command to run the program to view the output and return value.

---

2) **Task 2:** Modified program to return 1 and observed different exit status.

Ans:

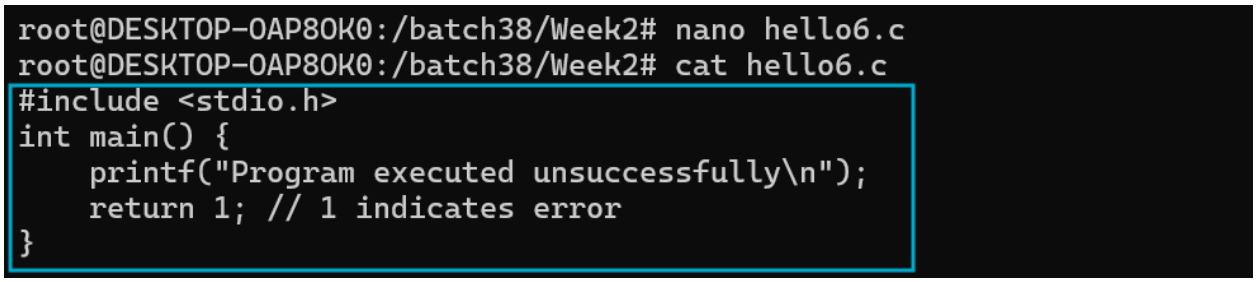


Figure 16: C language program modified to return 1 and print "Program executed unsuccessfully".

```
root@DESKTOP-OAP8OK0:/batch38/Week2# gcc -E hello6.c -o hello6.i
root@DESKTOP-OAP8OK0:/batch38/Week2# gcc -S hello6.i -o hello6.s
root@DESKTOP-OAP8OK0:/batch38/Week2# gcc -c hello6.s -o hello6.o
root@DESKTOP-OAP8OK0:/batch38/Week2# gcc hello6.o -o hello6
root@DESKTOP-OAP8OK0:/batch38/Week2# ./hello6
Program executed unsuccessfully
root@DESKTOP-OAP8OK0:/batch38/Week2# echo $?
1
```

Figure 17: Linux command to view the output and return value that was updated.

---

3) **Task 3**: Created program accepting user input and displaying it.

Ans:

```
root@DESKTOP-OAP8OK0:/batch38# touch hello1.c
root@DESKTOP-OAP8OK0:/batch38# ls
hello  hello.c  hello.i  hello.o  hello.s  hello1.c
root@DESKTOP-OAP8OK0:/batch38# nano hello1.c
root@DESKTOP-OAP8OK0:/batch38# cat hello1.c
#include <stdio.h>
int main() {
        int number;
        printf("Enter a number: ");
        scanf("%i", &number);
        printf("You entered: %i\n", number);
        return 0;
}
```

Figure 18: C language program to accept user input and display it.

```
root@DESKTOP-OAP8OK0:/batch38# ./hello1
Enter a number: 5
You entered: 5
root@DESKTOP-OAP8OK0:/batch38# echo $?
0
```

Figure 19: Linux command to view the output and return value.

---

4) **Task 4**: Executed all four compilation stages separately and documented output files.

Ans:

i) Compilation stages of task 1:

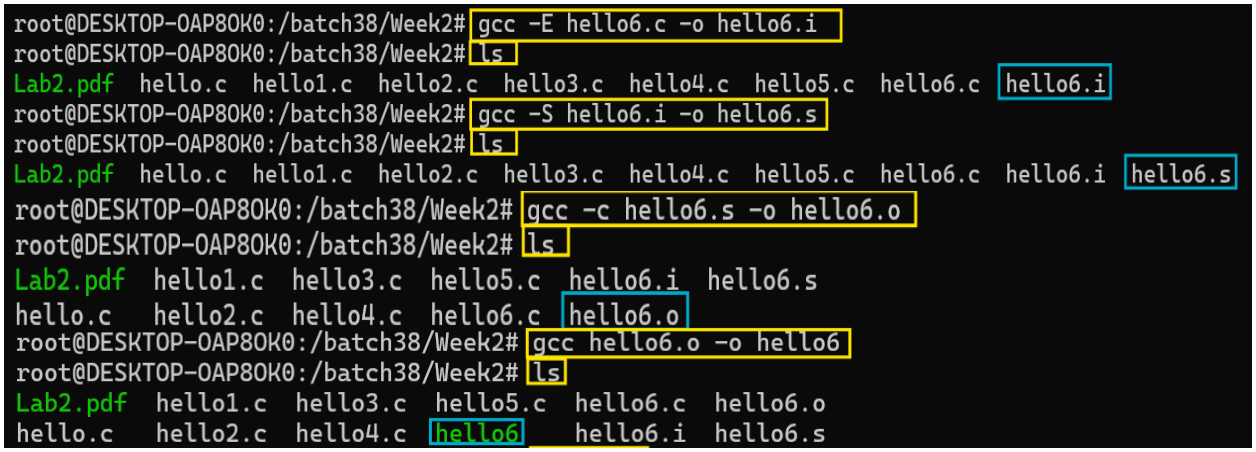


Figure 20: Linux command for 4 compilation stages with its output files. Linux command to run the program to view the output and return value.

ii) Compilation stages of task 3:

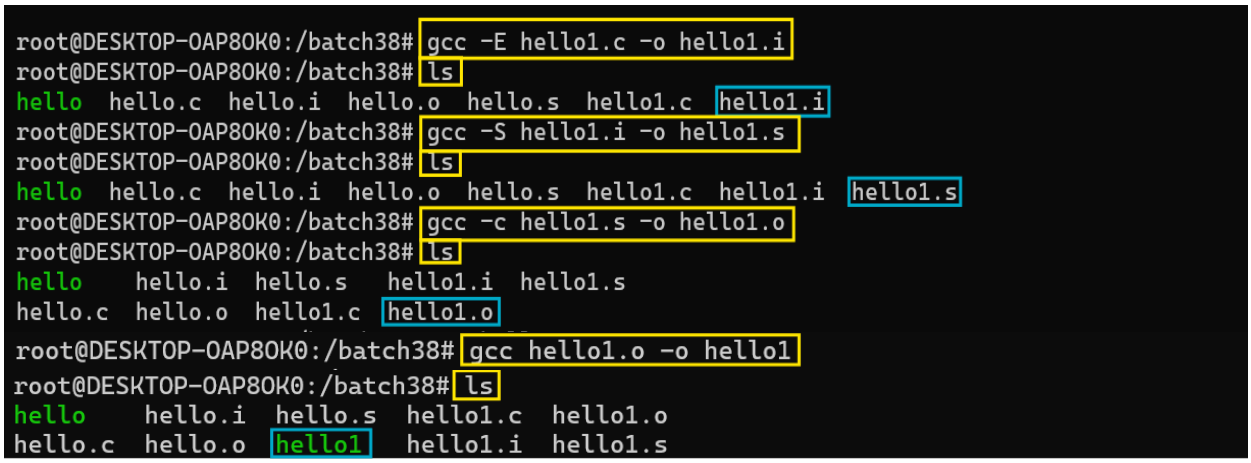


Figure 21: Linux command for 4 compilation stages with its output files. Linux command to run the program to view the output and return value.

5) **Task 5**: Compiled and ran program returning integer value 50

Ans:

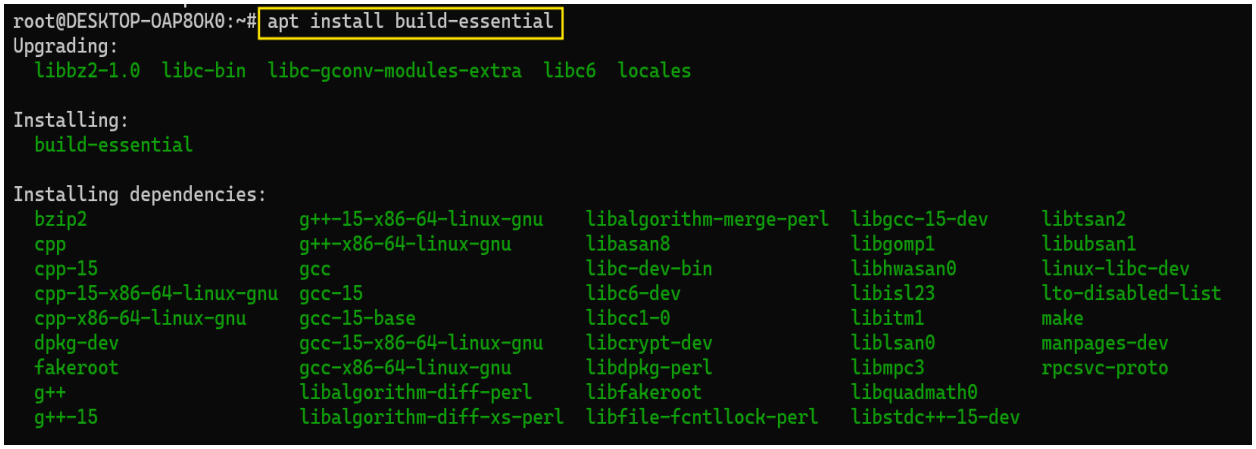
```
root@DESKTOP-OAP8OK0:/batch38# touch hello2.c
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c
hello.c  hello.o  hello1   hello1.i  hello1.s
root@DESKTOP-OAP8OK0:/batch38# nano hello2.c
root@DESKTOP-OAP8OK0:/batch38# cat hello2.c
#include <stdio.h>
int main() {
        return 50;
}
```

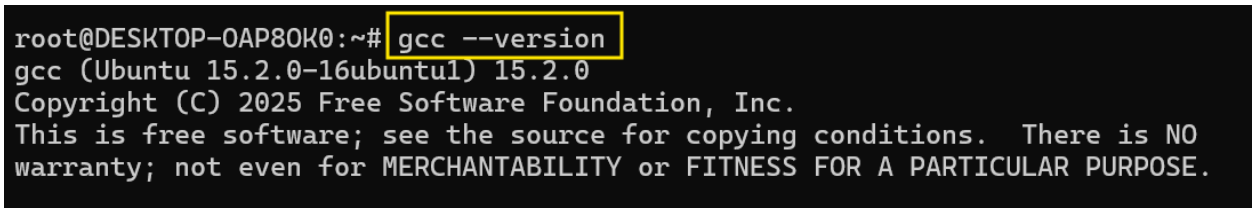
Figure 22: C language program that returns integer value 50.

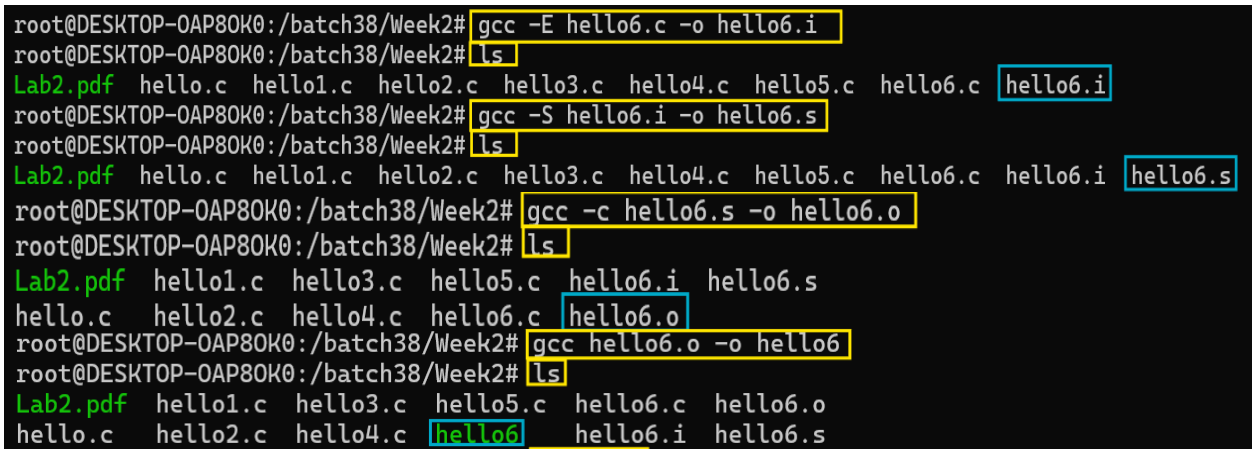
```
root@DESKTOP-OAP8OK0:/batch38# gcc -E hello2.c -o hello2.i
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c
hello.c  hello.o  hello1   hello1.i  hello1.s  hello2.i
root@DESKTOP-OAP8OK0:/batch38# gcc -S hello2.i -o hello2.s
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c  hello2.s
hello.c  hello.o  hello1   hello1.i  hello1.s  hello2.i
root@DESKTOP-OAP8OK0:/batch38# gcc -c hello2.s -o hello2.o
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c  hello2.o
hello.c  hello.o  hello1   hello1.i  hello1.s  hello2.i  hello2.s
root@DESKTOP-OAP8OK0:/batch38# gcc hello2.o -o hello2
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.o  hello1.c  hello1.s  hello2.i
hello.c  hello.s  hello1.i  hello2    hello2.o
hello.i  hello1   hello1.o  hello2.c  hello2.s
root@DESKTOP-OAP8OK0:/batch38# ./hello2
root@DESKTOP-OAP8OK0:/batch38# echo $?
50
```

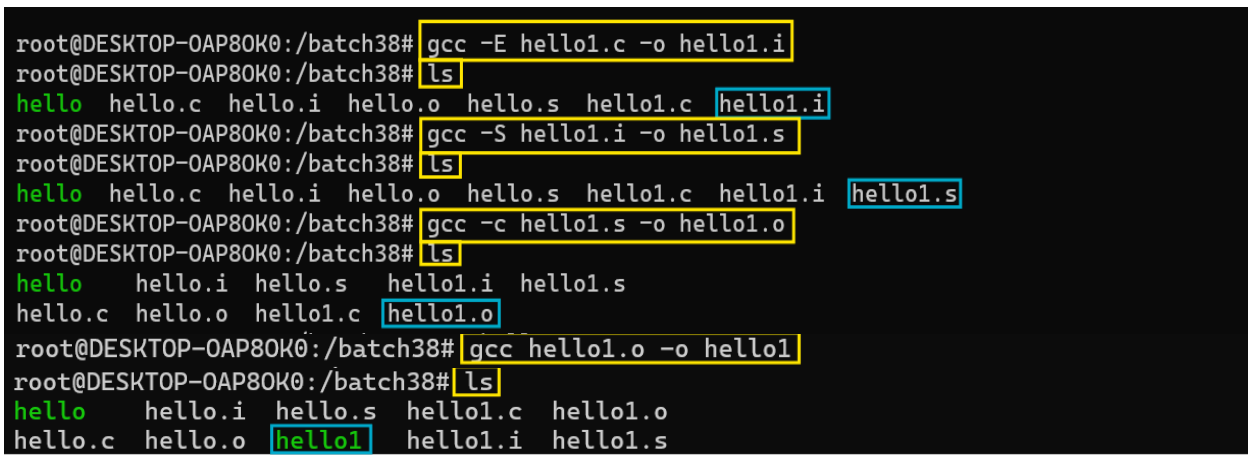
Figure 23: Linux command for 4 compilation stages with its output files. And linux command to run the program to view the output and return value.

Screenshot and evidence









```
root@DESKTOP-OAP8OK0:/batch38# gcc -E hello2.c -o hello2.i
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c
hello.c  hello.o  hello1   hello1.i  hello1.s  hello2.i
root@DESKTOP-OAP8OK0:/batch38# gcc -S hello2.i -o hello2.s
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c  hello2.s
hello.c  hello.o  hello1   hello1.i  hello1.s  hello2.i
root@DESKTOP-OAP8OK0:/batch38# gcc -c hello2.s -o hello2.o
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c  hello2.o
hello.c  hello.o  hello1   hello1.i  hello1.s  hello2.i  hello2.s
root@DESKTOP-OAP8OK0:/batch38# gcc hello2.o -o hello2
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.o  hello1.c  hello1.s  hello2.i
hello.c  hello.s  hello1.i  hello2    hello2.o
hello.i  hello1   hello1.o  hello2.c  hello2.s
```

```
root@DESKTOP-OAP8OK0:/batch38/Week2# ./hello6
Program executed successfully
root@DESKTOP-OAP8OK0:/batch38/Week2# echo $?
0
```

```
root@DESKTOP-OAP8OK0:/batch38/Week2# ./hello6
Program executed unsuccessfully
root@DESKTOP-OAP8OK0:/batch38/Week2# echo $?
1
```

```
root@DESKTOP-OAP8OK0:/batch38# ./hello1
Enter a number: 5
You entered: 5
root@DESKTOP-OAP8OK0:/batch38# echo $?
0
```

```
root@DESKTOP-OAP8OK0:/batch38# ./hello2
root@DESKTOP-OAP8OK0:/batch38# echo $?
50
```

Conclusion

This lab exercise provided hands-on experience with C programming fundamentals, including program structure, compilation process, and GCC usage. Understanding these concepts is essential for systems programming and operating system development. The four-stage compilation process demonstrates how high-level code is transformed into machine-executable instructions.